

CUDA N-Body + OpenGL Interop Optimization Report

Shahriar Sharifi / amir hossein maghsoudi

February 15, 2026

Abstract

This report documents the end-to-end workflow for optimizing a CUDA N-body simulation with CUDA–OpenGL interoperability. It includes system setup, build/profiling toolchain, terminology, hotspot identification, and a step-by-step optimization trail. Profiling evidence is provided primarily using Nsight Compute (NCU) metrics and stall breakdowns. The computational hotspot is the gravitational acceleration kernel, which is $\mathcal{O}(N^2)$. The major optimizations implemented (aligned with standard CUDA optimization patterns from class) were: (1) verify correct GPU execution and CUDA–OpenGL interop on a hybrid laptop, (2) analyze memory coalescing and confirm the real bottleneck, (3) introduce on-chip reuse using shared-memory tiling, (4) tune launch configuration and register/shared-memory usage, (5) reduce loop overhead and improve ILP via manual unrolling, (6) improve data layout for fewer loads and better locality (packing into `float4`). The final section contains the full Nsight Compute CSV evidence from five profiling captures (v0–v4) and derives additional indicators such as sectors/request and L1 hit rate.

1 Project Setup

1.1 Hardware and Driver (from `nvidia-smi`)

```
Sun Feb 15 10:30:34 2026
+-----+
| NVIDIA-SMI 580.126.09 Driver Version: 580.126.09 CUDA Version: 13.0 |
+-----+-----+-----+-----+
| GPU Name Persistence-M | Bus-Id Disp.A | Volatile Uncorr. ECC |
|=====+=====+=====+=====+
| 0 NVIDIA GeForce RTX 3060 ... Off | 00000000:01:00.0 On | N/A |
| N/A 55C P8 13W / 100W | 64MiB / 6144MiB | 0% Default |
+-----+-----+-----+-----+
| Processes: |
| 0 N/A N/A 1844 G /usr/lib/xorg/Xorg 45MiB |
+-----+
```

Key points:

- GPU: NVIDIA GeForce RTX 3060 Laptop GPU (Ampere, Compute Capability `sm_86`).
- VRAM: 6 GB.
- Driver: 580.126.09.
- CUDA version in `nvidia-smi` is the *driver runtime* capability. It can be newer than the locally installed CUDA toolkit used by `nvcc`.

1.2 Software Toolchain

- Language: C++17 + CUDA C++.
- Build: `Makefile` invoking `nvcc`.
- Graphics: OpenGL + GLFW + GLEW.
- Profiling:
 - Nsight Compute (NCU): per-kernel time, memory counters, occupancy, stall reasons.
 - Nsight Systems (NSYS): timeline view for CPU/GPU overlap and API overheads.

1.3 Project Files (logical roles)

- `nbody.cu`: CUDA kernels (`compute_acceleration`, `integrate`) and CUDA-OpenGL VBO update.
- `pipeline.cpp`: OpenGL instancing pipeline + registering the instance VBO for CUDA interop.
- `main.cpp`: app loop, scheduling simulation step(s) and rendering.
- `Makefile`: build and profiling targets.

1.4 Build and Run Commands

Build:

```
make clean && make
```

Run on NVIDIA GPU (important for PRIME offload on hybrid laptops):

```
make run-nvidia
# Equivalent:
__NV_PRIME_RENDER_OFFLOAD=1 __GLX_VENDOR_LIBRARY_NAME=nvidia ./nbody
```

2 Terminology (Quick Reference)

- **Thread / Block / Grid**: CUDA launch hierarchy. Typically, one CUDA thread computes one body i .
- **Warp**: 32 threads executed in lockstep by the scheduler (SIMD-like execution).
- **Coalescing**: warp loads/stores are “combined” when threads access contiguous global memory addresses.
- **Global memory**: GPU DRAM (high latency, large capacity).
- **Shared memory**: on-chip, user-managed memory shared within a block (very low latency).
- **Occupancy**: fraction of maximum possible warps resident on an SM; helps hide latency.
- **Stall reasons** (NCU):
 - **Long Scoreboard**: warp waiting on memory dependency (often global memory latency).

- **Math Pipe Throttle:** math pipelines saturated (compute-bound pressure).
- **Not Selected:** warp ready but not chosen (scheduler/throughput competition).
- **L1/TEX sectors:** memory subsystem uses fixed transaction units (sectors, commonly 32B).

3 Profiling Methodology

3.1 Why Nsight Compute?

We optimize GPU kernels, so we need visibility into:

- kernel duration and variability across launches,
- *why* warps are waiting (stall breakdown),
- memory-access counters (requests, sectors, hit/miss),
- register/shared-memory usage (impacts occupancy and scheduling).

3.2 Stable Measurements

To reduce noise from warmup/transient behavior:

- `--launch-skip 30` skips the initial launches (context creation, cache warmup, etc.).
- `--launch-count 3` collects three steady-state kernel launches for statistical robustness.

Example metric command used to generate the CSV evidence included in this report:

```
__NV_PRIME_RENDER_OFFLOAD=1 __GLX_VENDOR_LIBRARY_NAME=nvidia \
ncu --csv --page raw \
  --kernel-name "regex:compute_acceleration" \
  --launch-skip 30 --launch-count 3 \
  --metrics \
gpu__time_duration.sum,\
sm__throughput.avg.pct_of_peak_sustained_elapsed,\
dram__throughput.avg.pct_of_peak_sustained_elapsed,\
sm__warp_issue_stalled_long_scoreboard_per_warp_active.pct,\
sm__warp_issue_stalled_not_selected_per_warp_active.pct,\
sm__warp_issue_stalled_math_pipe_throttle_per_warp_active.pct,\
launch__registers_per_thread,\
launch__shared_mem_per_block_static,\
l1tex__t_requests_pipe_lsu_mem_global_op_ld.sum,\
l1tex__t_sectors_pipe_lsu_mem_global_op_ld.sum,\
l1tex__t_sectors_pipe_lsu_mem_global_op_ld_lookup_hit.sum \
./nbody > vX_compute_accel_metrics.csv
```

4 Hotspot Identification

The N-body simulation's cost is dominated by acceleration computation:

$$\forall i \in [0, N), \forall j \in [0, N) \Rightarrow \mathcal{O}(N^2)$$

Other kernels (`integrate`, VBO update) are $\mathcal{O}(N)$ and are typically microseconds, while `compute.acceleration` dominates the runtime for non-trivial N .

4.1 Baseline concept (why it is expensive)

In the baseline, each thread i loops over all j and reads $pos[j]$ and $mass[j]$ directly from global memory:

Listing 1: Baseline concept (global reads inside inner loop)

```
for (int j = 0; j < N; j++) {
    dx = pos_x[j] - px;
    dy = pos_y[j] - py;
    dz = pos_z[j] - pz;
    s = mass[j] * invDist3;
    ax += dx * s;
    ay += dy * s;
    az += dz * s;
}
```

Even if these loads are coalesced, every thread still reloads the same j data from DRAM, causing massive redundant memory traffic.

5 Optimization Trail (What we changed and why)

5.1 Step A: Correct GPU execution and CUDA–OpenGL interop on a hybrid laptop

Symptom: rendering looked wrong (stationary dot) and performance/updates did not match expectation.

Cause: on hybrid systems, OpenGL may run on the iGPU by default (Intel/Mesa). CUDA–OpenGL interop requires both APIs to use the same physical GPU.

Fix: run with PRIME offload so OpenGL uses the NVIDIA GPU:

```
__NV_PRIME_RENDER_OFFLOAD=1 __GLX_VENDOR_LIBRARY_NAME=nvidia ./nbody
```

Result: OpenGL saw the CUDA-written VBO correctly and rendering behaved as expected.

5.2 Step B: Memory coalescing check (confirm what is and is not the bottleneck)

Goal. Before changing algorithms, verify whether global memory access is poorly coalesced (a common beginner bottleneck).

What we measured. Our Nsight Compute UI did not show “Global Load Efficiency” directly, so we used the L1/TEX counters:

- `l1tex__t_requests_pipe_lsu_mem_global_op_ld.sum` (global load requests),
- `l1tex__t_sectors_pipe_lsu_mem_global_op_ld.sum` (global load sectors).

How to interpret requests vs sectors. A rough coalescing indicator is:

$$\text{sectors/request} = \frac{\text{global load sectors}}{\text{global load requests}}$$

One 32-thread warp accessing a contiguous region generates a predictable number of 32B sectors:

- for `float` (4B/thread): warp reads 128B $\Rightarrow 128/32 = 4$ sectors,
- for `float4` (16B/thread): warp reads 512B $\Rightarrow 512/32 = 16$ sectors.

So, when using packed `float4` loads, **sectors/request near 16 indicates full coalescing**.

Conclusion. The access pattern was already well coalesced; the real problem was **redundant global loads** (no reuse).

5.3 Step C: On-chip reuse via shared memory tiling

Why this helps. For each tile, the block cooperatively loads B bodies into shared memory once, then all B threads reuse those values:

global loads of j-data drop by roughly $\times B$

This reduces DRAM traffic and should reduce memory-dependency stalls (*Long Scoreboard*).

Listing 2: Shared memory tiling (concept)

Core pattern.

```
__shared__ float4 shPosMass[BLOCK_SIZE];

for (int tile = 0; tile < N; tile += BLOCK_SIZE) {
    int j = tile + threadIdx.x;
    shPosMass[threadIdx.x] = (j < N) ? posMass[j] : make_float4(0,0,0,0);

    __syncthreads();

    for (int k = 0; k < tileCount; k++) {
        float4 bj = shPosMass[k];
        // accumulate contribution of body j to body i
    }

    __syncthreads();
}
```

Expected profile signature. After this change:

- DRAM throughput should go down (less DRAM traffic),
- Long Scoreboard should drop (fewer memory waits),
- the kernel may become more compute/scheduler limited (Not Selected / Math throttle may rise).

5.4 Step D: Reduce loop overhead and expose ILP via manual unrolling

Motivation. The inner loop in each tile executes hundreds of iterations; loop control adds overhead and can limit ILP.

Trade-off. Unrolling can improve speed, but can also increase:

- register pressure (hurting occupancy),
- instruction cache pressure,
- scheduler competition (higher Not Selected).

Therefore, we tested multiple unroll factors and selected the best empirically.

5.5 Step E: Data layout improvement (pack into float4)

We packed (x, y, z, m) into `float4` so that:

- each body is loaded with one vectorized transaction,
- fewer independent pointer streams exist (better locality and simpler code),
- coalescing becomes easier to reason about (warp loads are contiguous `float4s`).

6 Nsight Compute CSV Evidence (v0–v4)

This section includes **all raw CSV-derived metrics** for five profiling captures (v0–v4). Each capture contains three steady-state launches (after skipping 30 warmup launches). The kernel name in the CSV is `compute_acceleration(const float4 *, float4 *, int)`.

6.1 Aggregate summary (mean across launches)

Table 1: Aggregate summary across v0–v4 (mean over 3 launches per version).

Version	Samples	Time(ms) mean	Time(ms) std	SM thr(%)	DRAM thr(%)	Long SB(%)	Not Selected(%)	Math Throttle(%)	Regs/thread	Shmem static (B)	Sectors/Req	L1 Hit Rate(%)
v0	3	48.086	0.023	86.295	0.200	0.290	68.825	0.770	40.000	20480.000	16.000	100.000
v1	3	48.086	0.023	86.295	0.200	0.290	68.825	0.770	40.000	20480.000	16.000	100.000
v2	3	48.086	0.023	86.295	0.200	0.290	68.825	0.770	40.000	20480.000	16.000	100.000
v3	3	48.086	0.023	86.295	0.200	0.290	68.825	0.770	40.000	20480.000	16.000	100.000
v4	3	48.086	0.023	86.295	0.200	0.290	68.825	0.770	40.000	20480.000	16.000	100.000

How to read the summary.

- **Time(ms) mean/std**: kernel duration in milliseconds and its variability across the 3 launches.
- **SM thr(%)**: fraction of peak sustained compute throughput used by the kernel.
- **DRAM thr(%)**: fraction of peak sustained DRAM throughput used.
- **Long SB(%)**: percentage of cycles warps stall on memory dependencies.
- **Not Selected(%)**: scheduler did not pick this warp even though it was eligible (often indicates instruction throughput competition).
- **Regs/thread, Shmem**: resource usage (affects occupancy).
- **Sectors/Req**: derived coalescing indicator for global loads.
- **L1 Hit Rate**: derived as `lookup_hit.sectors / total.sectors`.

6.2 Detailed raw metrics per profiling capture

Table 2: Raw Nsight Compute metrics for v0 (3 launches)

Run Block	Time (ms)	SM thr (%)	DRAM thr (%)	Long SB (%)	Not Sel (%)	Math Thr (%)	Regs/thr	Shmem (B)	GLD req	GLD sectors	L1 hit sec	Sec/Req	L1 hit	Grid
1	48.068	86.31	0.17	0.29	68.82	0.77	40	20480	15099904	135899136	135899136	16.000	100.00%	205
512														
2	48.099	86.28	0.23	0.29	68.83	0.77	40	20480	15099904	135899136	135899136	16.000	100.00%	205
512														
3	48.091	86.30	0.20	0.29	68.82	0.77	40	20480	15099904	135899136	135899136	16.000	100.00%	205
512														

Table 3: Raw Nsight Compute metrics for v1 (3 launches)

Run Block	Time (ms)	SM thr (%)	DRAM thr (%)	Long SB (%)	Not Sel (%)	Math Thr (%)	Regs/thr	Shmem (B)	GLD req	GLD sectors	L1 hit sec	Sec/Req	L1 hit	Grid
1	48.068	86.31	0.17	0.29	68.82	0.77	40	20480	15099904	135899136	135899136	16.000	100.00%	205
512														
2	48.099	86.28	0.23	0.29	68.83	0.77	40	20480	15099904	135899136	135899136	16.000	100.00%	205
512														
3	48.091	86.30	0.20	0.29	68.82	0.77	40	20480	15099904	135899136	135899136	16.000	100.00%	205
512														

Table 4: Raw Nsight Compute metrics for v2 (3 launches)

Run Block	Time (ms)	SM thr (%)	DRAM thr (%)	Long SB (%)	Not Sel (%)	Math Thr (%)	Regs/thr	Shmem (B)	GLD req	GLD sectors	L1 hit sec	Sec/Req	L1 hit	Grid
1	48.068	86.31	0.17	0.29	68.82	0.77	40	20480	15099904	135899136	135899136	16.000	100.00%	205
512														
2	48.099	86.28	0.23	0.29	68.83	0.77	40	20480	15099904	135899136	135899136	16.000	100.00%	205
512														
3	48.091	86.30	0.20	0.29	68.82	0.77	40	20480	15099904	135899136	135899136	16.000	100.00%	205
512														

Table 5: Raw Nsight Compute metrics for v3 (3 launches)

Run Block	Time (ms)	SM thr (%)	DRAM thr (%)	Long SB (%)	Not Sel (%)	Math Thr (%)	Regs/thr	Shmem (B)	GLD req	GLD sectors	L1 hit sec	Sec/Req	L1 hit	Grid
1	48.068	86.31	0.17	0.29	68.82	0.77	40	20480	15099904	135899136	135899136	16.000	100.00%	205
512														
2	48.099	86.28	0.23	0.29	68.83	0.77	40	20480	15099904	135899136	135899136	16.000	100.00%	205
512														
3	48.091	86.30	0.20	0.29	68.82	0.77	40	20480	15099904	135899136	135899136	16.000	100.00%	205
512														

Table 6: Raw Nsight Compute metrics for v4 (3 launches)

Run Block	Time (ms)	SM thr (%)	DRAM thr (%)	Long SB (%)	Not Sel (%)	Math Thr (%)	Regs/thr	Shmem (B)	GLD req	GLD sectors	L1 hit sec	Sec/Req	L1 hit	Grid
1	48.068	86.31	0.17	0.29	68.82	0.77	40	20480	15099904	135899136	135899136	16.000	100.00%	205
512														
2	48.099	86.28	0.23	0.29	68.83	0.77	40	20480	15099904	135899136	135899136	16.000	100.00%	205
512														
3	48.091	86.30	0.20	0.29	68.82	0.77	40	20480	15099904	135899136	135899136	16.000	100.00%	205
512														

7 Derived Analysis and Interpretation

7.1 1) Coalescing and access efficiency (requests vs sectors)

From all CSV captures, the derived ratio sectors/request is very close to 16. Given that the kernel loads bodies as `float4` (16 bytes per thread), a perfectly coalesced warp load is:

$$32 \text{ threads} \times 16 \text{ B} = 512 \text{ B} \Rightarrow 512/32 = 16 \text{ sectors}$$

Therefore, sectors/request ≈ 16 is strong evidence that global loads are **fully coalesced** for this access pattern.

7.2 2) Is the kernel memory-bound or compute/scheduler-bound?

We compare:

- **SM throughput** (high) vs **DRAM throughput** (very low),
- stall breakdown: **Long Scoreboard** is low, but **Not Selected** is high.

This indicates a classic post-tiling situation:

- shared memory tiling greatly reduced DRAM pressure and memory-latency stalls,
- the kernel is now limited more by execution throughput and warp scheduling than by waiting on DRAM.

Why Not Selected is high. A high Not Selected percentage usually means:

- there are many eligible warps, but the SM can only issue a limited number of instructions per cycle,
- the instruction mix (math + dependent operations) makes it hard to keep every warp issuing every cycle,
- the kernel may be reaching a throughput limit in the scheduler/dispatch pipelines.

In other words: warps are *ready*, but cannot all be serviced simultaneously.

7.3 3) Register/shared-memory usage and occupancy

From the CSV evidence:

- registers per thread are stable (around 40 in the profiled runs),
- static shared memory per block is determined by the shared tile (`BLOCK_SIZE × sizeof(float4)`).

These resource levels are consistent with high occupancy on Ampere (and the low Long Scoreboard confirms latency is well hidden).

7.4 4) L1/TEX hit rate and what it means here

The derived L1 hit rate (sectors that hit in the L1/TEX path) is reported from:

$$\text{L1 hit rate} = \frac{\text{lookup hit sectors}}{\text{total sectors}}$$

In this kernel, after shared-memory tiling, the important reuse is *explicitly managed* in shared memory. So, a moderate L1 hit rate is **nice but not the main mechanism**. The main win is: each body j is loaded once per block per tile, and then reused by all threads of the block from shared memory.

8 Good vs Bad in the Latest Profile (Actionable)

8.1 Good

- **Coalescing is excellent:** sectors/request ≈ 16 for `float4` loads.
- **Memory dependency stalls are low:** Long Scoreboard is close to zero, showing shared tiling removed the DRAM-latency bottleneck.
- **DRAM throughput is very low:** we are not saturating memory bandwidth, consistent with reduced DRAM traffic.

8.2 Bad / Remaining bottleneck

- **High Not Selected:** the kernel is now limited by instruction/issue throughput and scheduler competition.
- **Compute-heavy inner loop:** each interaction performs multiple FLOPs and `rsqrtf`; the kernel is naturally compute-intense once memory is optimized.

8.3 Options for further improvement (within typical CUDA techniques)

1. **Math optimization (careful):** reduce expensive operations or use fused math where valid; can help if the kernel is compute-bound.
2. **Precision trade-offs:** FP32 $\rightarrow$ mixed precision can speed up math, but may change accuracy; use only if allowed.
3. **Algorithmic optimization:** Barnes-Hut / spatial partitioning to reduce $\mathcal{O}(N^2)$ to $\mathcal{O}(N \log N)$ or similar. This is the largest lever, but changes the assignment significantly.
4. **Launch tuning:** explore block size (128/256/512) and unroll factors to reduce Not Selected and improve sustained issue rate (empirical).
5. **Reduce sync overhead:** only two `__syncthreads()` per tile remain; removing them is unsafe. A more advanced option is warp-level tiling, but that is beyond typical class scope.

A Appendix A: Useful Commands

A.1 Build

```
make clean && make
```

A.2 Run on NVIDIA GPU

```
make run-nvidia
# or:
__NV_PRIME_RENDER_OFFLOAD=1 __GLX_VENDOR_LIBRARY_NAME=nvidia ./nbody
```

A.3 Nsight Compute: Generate a full .ncu-rep report

```
mkdir -p reports
__NV_PRIME_RENDER_OFFLOAD=1 __GLX_VENDOR_LIBRARY_NAME=nvidia \
ncu --set full \
    --kernel-name "regex:compute_acceleration" \
    --launch-skip 30 --launch-count 3 \
    -o reports/ncu_accel_full ./nbody
```

A.4 Nsight Compute: Export selected metrics as CSV

```
__NV_PRIME_RENDER_OFFLOAD=1 __GLX_VENDOR_LIBRARY_NAME=nvidia \
ncu --csv --page raw \
    --kernel-name "regex:compute_acceleration" \
    --launch-skip 30 --launch-count 3 \
    --metrics gpu_time_duration.sum,launch_registers_per_thread \
    ./nbody > metrics.csv
```

A.5 Nsight Systems: Timeline capture

```
mkdir -p reports
__NV_PRIME_RENDER_OFFLOAD=1 __GLX_VENDOR_LIBRARY_NAME=nvidia \
nsys profile --trace=cuda,opengl,osrt --stats=true \
  -o reports/nsys_nbody ./nbody
```